WY 4-3-1 Count Simple

题目描述

我们给出一个对整数做“快速排序“的C++程序。

可以看出, 它实现成了一个具有稳定性的排序,并且在待排序的数据过短的时候,会改用插入排序来减小常数。

但缺陷在于，选取轴点的时候，采用了比较简单的方式, 因此极易退化。

在程序运行过程中，维护了一个变量“count”，它用于估计程序执行了多少次比较、交换操作。

现在，将count初始化为0，给出array[]的内容，给出LIMIT的数值，请你给出按照下面的程序执行quicksort(array, 0, n)之后，count的最终数值。

你可以不使用下面的程序来计算count数值，但计算出来的数值应当和下面的程序计算出来的一致。

为了简单起见，我们将LIMIT的数值固定为1.

```
#include<cstdio>
#include<cstdlib>
inline void swap(int &a, int &b){
    int c = a; a = b; b = c;
}
int data[500000];
int buffer[500000];
unsigned long long count = 0;
int n,LIMIT;
void insertionSort(int array[],int L, int R){
    for(int i=L+1;i<R;++i){
        for(int j=i;j>L;--j){
            if(array[j]<array[j-1]){
                swap(array[j],array[j-1]);
                count++;
            }else{
                break;
            }
        }
    }
}
void quicksort(int array[], int buffer[], int L, int R){// stable + inplace  + O(n)?
    if(R-L<=LIMIT){
        insertionSort(array,L,R);//merge sort to calculate how many swaps happened
        return;
    }
    count += （R-L)*2;
    int pivot = array[L];
    int k = L;
    for(int i=L+1;i<R;++i){
        if(array[i] < pivot){
            buffer[k] = array[i];
            k+=1;
        }
    }
    int mid = k;
    buffer[mid] = pivot;
    k+=1;
    for(int i=L+1;i<R;++i){
        if(array[i]>=pivot){
            buffer[k] = array[i];
            k+=1;
        }
    }
    for(int i=L;i<R;++i){
        array[i] = buffer[i];
    }
    quicksort(array,buffer,L,mid);
    quicksort(array,buffer,mid+1,R);
}

int main(){
    scanf("%d %d",&n,&LIMIT);
    for(int i=0;i<n;++i){
```

```
        scanf("%d",&data[i]);
    }
    quicksort(data,buffer,0,n);
    for(int i=0;i<n-1;++i){
        if(data[i]>data[i+1])printf("%d failed",i);
    }
    printf("%llu\n", count);
    return 0;
}
```

输入格式

第一行两个数字，n和LIMIT。

第二行n个数字，表示array的内容。

输出格式

一个数字，表示count的最终数值。

输入样例

```
5 1
5 4 3 2 1
```

输出样例

```
28
```

样例解释

28 = 5x2 + 4x2 + 3x2 + 2x2

数据范围

0< n <= 500000，LIMIT=1

n个数字都在int范围内

时间：1 sec

空间：256 MB

本题允许使用STL。

OJ 测试时会调大栈空间限制避免导致segmentation fault。本地测试可以使用 ulimit -s 32768 命令调大栈空间限制。

UI powered by Twitter Bootstrap (http://getbootstrap.com/).
Tsinghua Online Judge is designed and coded by Li Ruizhe.
For all suggestions and bug reports, contact oj[at]liruizhe[dot]org.